

Лабораторная работа №1

Задание 1. Методы.

Задача 1. Дробная часть

Текст задачи

Дана сигнатура функции: `double fraction (double x);`

Необходимо реализовать функцию таким образом, чтобы она возвращала только дробную часть числа x . Подсказка: вещественное число может быть преобразовано к целому путем отбрасывания дробной части.

Пример:

$x=5,25$

результат: $0,25$

Алгоритм решения

Функция `double Fraction(double x)`:

1. Число x приводится к типу `int` оператором `static_cast<int>(x)`. При таком приведении дробная часть отбрасывается (усечение в сторону нуля), и в переменной `integer_part` остаётся целая часть числа.
2. Дробная часть вычисляется вычитанием целой части из исходного числа: $x - \text{integer_part}$. Для $x = 5.25$ целая часть равна 5, результат — 0.25. Для отрицательных чисел знак сохраняется ($-3.75 \rightarrow -0.75$).

Функция `main`: выводит приглашение к вводу, читает `double x` и проверяет ввод (чтение прошло успешно и в строке нет лишних символов). При ошибке выводится сообщение [ОШИБКА] и программа завершается с кодом 1. Иначе вызывается `Fraction(x)` и результат выводится на экран.

Тестирование

```
• wakezup@pc:~/Labs/Lab1$ ./1_1
  [ВВОД] Введите число: 5.25
  [РЕЗУЛЬТАТ] Дробная часть числа 5.25 равна 0.25
○ wakezup@pc:~/Labs/Lab1$
• wakezup@pc:~/Labs/Lab1$ ./1_1
  [ВВОД] Введите число: -2.25
  [РЕЗУЛЬТАТ] Дробная часть числа -2.25 равна -0.25
○ wakezup@pc:~/Labs/Lab1$
⊙ wakezup@pc:~/Labs/Lab1$ ./1_1
  [ВВОД] Введите число: abc
  [ОШИБКА] Введенное значение не является числом!
```

Задача 3. Букву в число

Текст задачи

Дана сигнатура функции: `int charToNum (char x);`

Функция принимает символ `x`, который представляет собой один из “0 1 2 3 4 5 6 7 8 9”. Необходимо реализовать функцию таким образом, чтобы она преобразовывала символ в соответствующее число. Подсказка: код символа ‘0’ — это число 48.

Пример:

`x='3'`

результат: 3

Алгоритм решения

Функция `int CharToNum(char x):`

1. Коды цифр в таблице ASCII идут подряд: '0' — 48, '1' — 49, ..., '9' — 57.
2. Поэтому числовое значение цифры равно разности кодов `static_cast<int>(x) - static_cast<int>('0')`. Например, для '3' (код 51) получаем $51 - 48 = 3$.

Функция `main`: читает один символ `x` и проверяет ввод. Ввод считается корректным, если символ прочитан успешно, в строке нет других символов (поэтому ввод 12 отвергается) и выполняется условие `'0' <= x <= '9'`. При нарушении любого условия выводится сообщение об ошибке и программа завершается с кодом 1. Иначе выводится результат `CharToNum(x)`.

Тестирование

```
• wakezup@pc:~/labs/lab1$ ./1_3
[ВВОД] Введите цифру: 3
[РЕЗУЛЬТАТ] Цифра 3 соответствует числу 3
• wakezup@pc:~/labs/lab1$ ./1_3
[ВВОД] Введите цифру: 9
[РЕЗУЛЬТАТ] Цифра 9 соответствует числу 9
• wakezup@pc:~/labs/lab1$ ./1_3
[ВВОД] Введите цифру: 12
[ОШИБКА] Введенное значение не является цифрой!
• wakezup@pc:~/labs/lab1$ ./1_3
[ВВОД] Введите цифру: а
[ОШИБКА] Введенное значение не является цифрой!
```

Задача 5. Двузначное

Текст задачи

Дана сигнатура функции: `bool is2Digits (int x);`

Необходимо реализовать функцию таким образом, чтобы она принимала число `x` и возвращала `true`, если оно двузначное.

Пример 1:

`x=32`

результат: `true`

Пример 2:

`x=516`

результат: `false`

Алгоритм решения

Функция `bool Is2Digits(int x):`

1. Число двузначное, если его модуль лежит в пределах от 10 до 99.
2. Функция возвращает значение логического выражения $(x \geq 10 \ \&\& \ x \leq 99)$ $\parallel (x \geq -99 \ \&\& \ x \leq -10)$. Вторая часть выражения учитывает отрицательные двузначные числа (например, `-45`).

Функция `main`: читает целое число `x` с проверкой ввода. Если ввод некорректен — сообщение об ошибке и `return 1`. Иначе по значению `Is2Digits(x)` выводится сообщение «является двузначным» или «не является двузначным».

Тестирование

```
• wakezup@pc:~/labs/lab1$ ./1_5
[ВВОД] Введите целое число: 32
[РЕЗУЛЬТАТ] Число 32 является двузначным.
• wakezup@pc:~/labs/lab1$ ./1_5
[ВВОД] Введите целое число: 516
[РЕЗУЛЬТАТ] Число 516 не является двузначным.
• wakezup@pc:~/labs/lab1$ ./1_5
[ВВОД] Введите целое число: -45
[РЕЗУЛЬТАТ] Число -45 является двузначным.
• wakezup@pc:~/labs/lab1$ ./1_5
[ВВОД] Введите целое число: abc
[ОШИБКА] Введенное значение не является целым числом!
```

Задача 7. Диапазон

Текст задачи

Дана сигнатура функции: `bool isInRange (int a, int b, int num);`

Функция принимает левую и правую границу (a и b) некоторого числового диапазона. Необходимо реализовать функцию таким образом, чтобы она возвращала `true`, если `num` входит в указанный диапазон (включая границы). Обратите внимание, что отношение a и b заранее неизвестно (неясно кто из них больше, а кто меньше)

Пример 1:

a=5 b=1 num=3

результат: `true`

Пример 2:

a=2 b=15 num=33

результат: `false`

Алгоритм решения

Функция `bool IsInRange(int a, int b, int number):`

1. Так как заранее неизвестно, какая из границ меньше, сначала проверяется условие `a > b`. Если оно выполнено, значения a и b меняются местами с помощью временной переменной `temp`. После этого a — левая (меньшая) граница, b — правая (большая).

2. Функция возвращает `number >= a && number <= b`. Операторы нестрогого сравнения обеспечивают включение границ в диапазон.

Функция `main`: сначала читает пару границ `a` и `b`, затем отдельным вводом — проверяемое число; каждый ввод сопровождается проверкой. В сообщении выводятся границы в том порядке, в котором их ввёл пользователь (в `main` значения передаются в функцию по значению, поэтому обмен внутри функции их не меняет).

Тестирование

```
• wakezup@pc:~/labs/lab1$ ./1_7
[ВВОД] Введите два числа (a и b): 5 1
[ВВОД] Введите число для проверки: 3
[РЕЗУЛЬТАТ] Число 3 находится в диапазоне от 5 до 1.
• wakezup@pc:~/labs/lab1$ ./1_7
[ВВОД] Введите два числа (a и b): 2 15
[ВВОД] Введите число для проверки: 33
[РЕЗУЛЬТАТ] Число 33 не находится в диапазоне от 2 до 15.
• wakezup@pc:~/labs/lab1$ ./1_7
[ВВОД] Введите два числа (a и b): a 1
[ОШИБКА] Введенные значения не являются целыми числами!
```

Задача 9. Равенство

Текст задачи

Дана сигнатура функции: `bool isEqual(int a, int b, int c);`

Необходимо реализовать функцию таким образом, чтобы она возвращала `true`, если все три полученных функцией числа равны

Пример 1:

`a=3 b=3 c=3`

результат: `true`

Пример 2:

`a=2 b=15 c=2`

результат: `false`

Алгоритм решения

Функция `bool IsEqual(int a, int b, int c)`:

1. Три числа равны, если первое равно второму и второе равно третьему.
2. Функция возвращает `a == b && b == c`. Сравнивать `a` с `c` отдельно не нужно: равенство транзитивно.

Функция `main`: читает три целых числа одной строкой с проверкой ввода и выводит сообщение «Все три числа равны» или «Числа не равны» в зависимости от результата функции.

Тестирование

```
• wakezup@pc:~/labs/lab1$ ./1_9
[ВВОД] Введите три числа: 3 3 3
[РЕЗУЛЬТАТ] Все три числа равны.
• wakezup@pc:~/labs/lab1$ ./1_9
[ВВОД] Введите три числа: 2 4 1
[РЕЗУЛЬТАТ] Числа не равны.
• wakezup@pc:~/labs/lab1$ ./1_9
[ВВОД] Введите три числа: 1 а 3
[ОШИБКА] Введенные значения не являются целыми числами!
❖ wakezup@pc:~/labs/lab1$
```

Задание 2. Условия

Задача 1. Модуль числа

Текст задачи

Дана сигнатура функции: `int abs (int x)`;

Необходимо реализовать функцию таким образом, чтобы она возвращала модуль числа `x` (если оно было положительным, то таким и остается, если оно было отрицательным – то необходимо вернуть его без знака минус).

Пример 1:

`x=5`

результат: 5

Пример 2:

`x=-3`

результат: 3

Алгоритм решения

Функция `int Abs(int x)`:

1. Если $x < 0$, функция возвращает $-x$ (число с противоположным знаком, то есть без минуса).
2. Иначе ($x \geq 0$) возвращается само x .

Функция `main`: читает целое число с проверкой ввода и выводит результат `Abs(x)`.

Тестирование

```
wakezur@pc:~/labs/lab1$ ./2_1
[ВВОД] Введите целое число: 5
[РЕЗУЛЬТАТ] Модуль числа 5 равен 5
wakezur@pc:~/labs/lab1$ ./2_1
[ВВОД] Введите целое число: -3
[РЕЗУЛЬТАТ] Модуль числа -3 равен 3
wakezur@pc:~/labs/lab1$ ./2_1
[ВВОД] Введите целое число: abc
[ОШИБКА] Введенное значение не является целым числом!
wakezur@pc:~/labs/lab1$
```

Задача 3. Тридцать пять

Текст задачи

Дана сигнатура функции: `bool is35 (int x)`;

Необходимо реализовать функцию таким образом, чтобы она возвращала `true`, если число x делится нацело на 3 или 5. При этом, если оно делится и на 3, и на 5, то вернуть надо `false`. Подсказка: оператор `%` позволяет получить остаток от деления.

Пример 1:

$x=5$

результат: `true`

Пример 2:

$x=8$

результат: `false`

Пример 3:

$x=15$

результат: `false`

Алгоритм решения

Функция `bool Is35(int x)`:

1. Внешнее условие `x % 3 == 0 || x % 5 == 0` проверяет, делится ли число хотя бы на одно из чисел 3 и 5 (остаток от деления равен нулю). Если не делится ни на одно, возвращается `false`.
2. Внутри внешнего условия проверяется, делится ли число на оба числа сразу: `x % 3 == 0 && x % 5 == 0`. Если да — возвращается `false` (по условию такие числа исключаются), иначе — `true`.

Таким образом, `true` возвращается только для чисел, которые делятся ровно на одно из чисел 3 и 5. Например, `5 → true`, `8 → false` (не делится ни на 3, ни на 5), `15 → false` (делится на оба).

Функция `main`: читает целое число с проверкой ввода и выводит соответствующее сообщение.

Тестирование

```
• wakezup@pc:~/labs/lab1$ ./2_3
[ВВОД] Введите целое число: 5
[РЕЗУЛЬТАТ] Число 5 делится на 3 или на 5, но не на оба.
• wakezup@pc:~/labs/lab1$ ./2_3
[ВВОД] Введите целое число: 8
[РЕЗУЛЬТАТ] Число 8 не делится на 3 или на 5, либо делится на оба.
• wakezup@pc:~/labs/lab1$ ./2_3
[ВВОД] Введите целое число: 15
[РЕЗУЛЬТАТ] Число 15 не делится на 3 или на 5, либо делится на оба.
• wakezup@pc:~/labs/lab1$ ./2_3
[ВВОД] Введите целое число: abc
[ОШИБКА] Введенное значение не является целым числом!
❖ wakezup@pc:~/labs/lab1$
```

Задача 5. Тройной максимум

Текст задачи

Дана сигнатура функции: `int max3 (int x, int y, int z)`;

Необходимо реализовать функцию таким образом, чтобы она возвращала максимальное из трех полученных функцией чисел. Подсказка: идеальное решение включает всего две инструкции `if` и не содержит вложенных `if`.

Пример 1:
x=5 y=7 z=7
результат: 7

Пример 2:
x=8 y=-1 z=4
результат: 8

Алгоритм решения

Функция `int Max3(int x, int y, int z)` построена как цепочка `if ... else if ... else` без вложенных условий (две инструкции `if`):

1. Если `x >= y && x >= z`, то `x` не меньше остальных — возвращается `x`.
2. Иначе, если `y >= x && y >= z`, возвращается `y`.
3. Иначе максимальным является `z`, и возвращается он.

Нестрогие сравнения (`>=`) корректно обрабатывают случай, когда несколько чисел равны между собой (например, 5, 7, 7).

Функция `main`: читает три целых числа с проверкой ввода и выводит максимальное из них.

Тестирование

```
wakeup@pc:~/labs/lab1$ ./2_5
[ВВОД] Введите три целых числа: 5 7 7
[РЕЗУЛЬТАТ] Максимальное число из 5, 7 и 7 равно 7.
wakeup@pc:~/labs/lab1$ ./2_5
[ВВОД] Введите три целых числа: 8 -1 4
[РЕЗУЛЬТАТ] Максимальное число из 8, -1 и 4 равно 8.
wakeup@pc:~/labs/lab1$ ./2_5
[ВВОД] Введите три целых числа: а 1 2
[ОШИБКА] Введенные значения не являются целыми числами!
wakeup@pc:~/labs/lab1$
```

Задача 7. Двойная сумма

Текст задачи

Дана сигнатура функции: `int sum2 (int x, int y);`

Необходимо реализовать функцию таким образом, чтобы она возвращала сумму чисел `x` и `y`. Однако, если сумма попадает в диапазон от 10 до 19, то надо вернуть число 20.

Пример 1:
x=5 y=7
результат: 20

Пример 2:
x=8 y=-1
результат: 7

Алгоритм решения

Функция `int Sum2(int x, int y)`:

1. Вычисляется сумма $sum = x + y$.
2. Если $sum \geq 10 \ \&\& \ sum \leq 19$ (сумма попала в диапазон от 10 до 19 включительно), функция возвращает 20.
3. Иначе возвращается сама сумма sum .

Функция `main`: читает два целых числа одной строкой с проверкой ввода и выводит результат вызова `Sum2(x, y)`.

Тестирование

```
• wakezup@pc:~/labs/lab1$ ./2_7
[ВВОД] Введите два целых числа: 5 7
[РЕЗУЛЬТАТ] Результат функции sum2(5, 7) равен 20.
• wakezup@pc:~/labs/lab1$ ./2_7
[ВВОД] Введите два целых числа: 8 -1
[РЕЗУЛЬТАТ] Результат функции sum2(8, -1) равен 7.
• wakezup@pc:~/labs/lab1$ ./2_7
[ВВОД] Введите два целых числа: 20 30
[РЕЗУЛЬТАТ] Результат функции sum2(20, 30) равен 50.
❖ wakezup@pc:~/labs/lab1$
```

Задача 9. День недели

Текст задачи

Дана сигнатура функции: `String day (int x)`;

Функция принимает число x , обозначающее день недели. Необходимо реализовать функцию таким образом, чтобы она возвращала строку, которая будет обозначать текущий день недели, где 1 — это понедельник, а 7 — воскресенье. Если число не от 1 до 7 то верните текст “это не день недели”. Вместо `if` в данной задаче используйте `switch`.

Пример:

x=5

результат: “пятница”

Алгоритм решения

Функция `std::string Day(int x)` построена на инструкции `switch` по значению `x`, как того требует условие:

1. Для каждого значения от 1 до 7 предусмотрена своя ветка `case`, которая сразу возвращает название дня: 1 — «Понедельник», 2 — «Вторник», ..., 7 — «Воскресенье». Так как каждая ветка завершается `return`, оператор `break` не нужен.
2. Ветка `default` выполняется для любого другого значения и возвращает текст «Это не день недели».

Функция `main`: читает целое число с проверкой ввода и выводит строку, возвращённую `Day(x)`.

Тестирование

```
[РЕЗУЛЬТАТ] Результат функции sum2(20, 30) равен 50.
• wakezup@pc:~/Labs/Lab1$ ./2_9
[ВВОД] Введите целое число: 5
[РЕЗУЛЬТАТ] Пятница
• wakezup@pc:~/Labs/Lab1$ ./2_9
[ВВОД] Введите целое число: 1
[РЕЗУЛЬТАТ] Понедельник
• wakezup@pc:~/Labs/Lab1$ ./2_9
[ВВОД] Введите целое число: 19
[РЕЗУЛЬТАТ] Это не день недели
• wakezup@pc:~/Labs/Lab1$ ./2_9
[ВВОД] Введите целое число: abc
[ОШИБКА] Введенное значение не является целым числом!
❖ wakezup@pc:~/Labs/Lab1$
```

Задание 3. Циклы

Задача 1. Числа подряд

Текст задачи

Дана сигнатура функции: `String listNums (int x);`

Необходимо реализовать функцию таким образом, чтобы она возвращала строку, в которой будут записаны все числа от 0 до x (включительно).

Пример:

x=5

результат: "0 1 2 3 4 5"

Алгоритм решения

Функция `std::string ListNums(int x)`:

1. Создаётся пустая строка `result`.
2. Цикл `for` перебирает `i` от 0 до x включительно (условие `i <= x`).
3. На каждой итерации число переводится в строку функцией `std::to_string(i)` и добавляется к `result`.
4. По завершении цикла строка возвращается.

Функция `main`: читает целое число с проверкой ввода и выводит строку, возвращённую `ListNums(x)`.

Тестирование

```
• wakezup@pc:~/labs/lab1$ ./3_1
[ВВОД] Введите целое число: 5
[РЕЗУЛЬТАТ] Список чисел от 0 до 5: 0 1 2 3 4 5
• wakezup@pc:~/labs/lab1$ ./3_1
[ВВОД] Введите целое число: 0
[РЕЗУЛЬТАТ] Список чисел от 0 до 0: 0
• wakezup@pc:~/labs/lab1$ ./3_1
[ВВОД] Введите целое число: abc
[ОШИБКА] Введенное значение не является целым числом!
❖ wakezup@pc:~/labs/lab1$
```

Задача 3. Четные числа

Текст задачи

Дана сигнатура функции: `String chet (int x)`;

Необходимо реализовать функцию таким образом, чтобы она возвращала строку, в которой будут записаны все четные числа от 0 до x (включительно). Подсказка для обеспечения качества кода: инструкцию `if` использовать не следует.

Пример:

x=9

результат: "0 2 4 6 8"

Алгоритм решения

Функция `std::string Chet(int x)`:

1. Создаётся пустая строка `result`.
2. Чтобы обойтись без `if`, чётность не проверяется: цикл `for` начинается с 0 и на каждой итерации увеличивает счётчик на 2 (`i += 2`), то есть перебирает только чётные числа. Условие продолжения — `i <= x`, поэтому `x` включается в результат, если оно чётное.
3. На каждой итерации число переводится в строку функцией `std::to_string(i)` и вместе с разделителем " " добавляется к `result`.
4. По завершении цикла строка возвращается.

В функции нет ни одной инструкции `if`, что соответствует подсказке в условии.

Функция `main`: читает целое число с проверкой ввода и выводит строку, возвращённую `Chet(x)`.

Тестирование

```
• wakezup@pc:~/labs/lab1$ ./3_3
[ВВОД] Введите целое число: 9
[РЕЗУЛЬТАТ] Четные числа от 0 до 9: 0 2 4 6 8
• wakezup@pc:~/labs/lab1$ ./3_3
[ВВОД] Введите целое число: 10
[РЕЗУЛЬТАТ] Четные числа от 0 до 10: 0 2 4 6 8 10
• wakezup@pc:~/labs/lab1$ ./3_3
[ВВОД] Введите целое число: abc
[ОШИБКА] Введенное значение не является целым числом!
• wakezup@pc:~/labs/lab1$ ./3_3
[ВВОД] Введите целое число: -1
[РЕЗУЛЬТАТ] Четные числа от 0 до -1:
❖ wakezup@pc:~/labs/lab1$
```

Задача 5. Длина числа

Текст задачи

Дана сигнатура функции: `int numLen (long x);`

Необходимо реализовать функцию таким образом, чтобы она возвращала количество знаков в числе x .

Подсказка:

`int y=123/10; // y будет иметь значение 12`

Пример:

`x=12567`

результат: 5

Алгоритм решения

Функция `int NumLen(long x)`:

1. Особый случай: если $x == 0$, возвращается 1, так как запись числа 0 состоит из одной цифры (цикл ниже для нуля не выполнялся бы ни разу).
2. Иначе заводится счётчик `length = 0`, и пока $x != 0$, выполняются два действия: $x /= 10$ (целочисленное деление на 10 отбрасывает последнюю цифру) и $length++$.
3. Когда число обращается в 0, в `length` находится количество отброшенных цифр, оно и возвращается.

Функция `main`: читает число типа `long` с проверкой ввода и выводит результат `NumLen(x)`.

Тестирование

```
● wakezup@pc:~/labs/lab1$ ./3_5
[ВВОД] Введите целое число: 12467
[РЕЗУЛЬТАТ] Количество цифр в числе 12467 равно 5.
● wakezup@pc:~/labs/lab1$ ./3_5
[ВВОД] Введите целое число: 0
[РЕЗУЛЬТАТ] Количество цифр в числе 0 равно 1.
● wakezup@pc:~/labs/lab1$ ./3_5
[ВВОД] Введите целое число: -3401
[РЕЗУЛЬТАТ] Количество цифр в числе -3401 равно 4.
⊙ wakezup@pc:~/labs/lab1$ ./3_5
[ВВОД] Введите целое число: abc
[ОШИБКА] Введенное значение не является целым числом!
❖ wakezup@pc:~/labs/lab1$
```

Задача 7. Квадрат

Текст задачи

Дана сигнатура функции: `void square (int x);`

Необходимо реализовать функцию таким образом, чтобы она выводила на экран квадрат из символов '*' размером x, у которого x символов в ряд и x символов в высоту.

Пример 1:

x=2

результат:

```
 **
 **
```

Пример 2:

x=4

результат:

```
****
****
****
****
```

Алгоритм решения

Функция `void Square(int x)` использует два вложенных цикла `for`:

1. Внешний цикл по `i` от 1 до `x` отвечает за строки квадрата.
2. Внутренний цикл по `j` от 1 до `x` выводит в текущей строке `x` символов '*' (после каждой звёздочки выводится пробел, чтобы фигура на экране выглядела ближе к квадрату).
3. После завершения внутреннего цикла выводится символ перевода строки '\n', и внешний цикл переходит к следующей строке.

Всего выводится x строк по x звёздочек.

Функция main: читает целое число с проверкой ввода. Дополнительно проверяется, что число положительное ($x > 0$): для нуля и отрицательных значений квадрат построить нельзя, и выводится сообщение об ошибке. Затем вызывается `Square(x)`.

Тестирование

```
• wakezup@pc:~/labs/lab1$ ./3_7
[ВВОД] Введите целое положительное число: 2
[РЕЗУЛЬТАТ] Квадрат из 2 строк и 2 столбцов:
* *
* *

• wakezup@pc:~/labs/lab1$ ./3_7
[ВВОД] Введите целое положительное число: 4
[РЕЗУЛЬТАТ] Квадрат из 4 строк и 4 столбцов:
* * * *
* * * *
* * * *
* * * *

⊗ wakezup@pc:~/labs/lab1$ ./3_7
[ВВОД] Введите целое положительное число: 0
[ОШИБКА] Введенное значение не является целым положительным числом!
❖ wakezup@pc:~/labs/lab1$
```

Задача 9. Правый треугольник

Текст задачи

Дана сигнатура функции: `void rightTriangle (int x);`

Необходимо реализовать функцию таким образом, чтобы она выводила на экран треугольник из символов ‘`*`’ у которого x символов в высоту, а количество символов в ряду совпадает с номером строки, при этом треугольник выровнен по правому краю. Подсказка: перед символами ‘`*`’ следует вывести необходимое количество пробелов.

Пример 1:

$x=3$

результат:

```
  *
 * *
* * *
```

Пример 2:

$x=4$

результат:

```
  *
 * *
* * *
* * * *
```

Алгоритм решения

Функция `void RightTriangle(int x)` содержит внешний цикл по строкам и два последовательных внутренних цикла:

1. Внешний цикл по i от 1 до x — номер текущей строки.
2. Первый внутренний цикл выполняется $x - i$ раз и выводит отступ. Чем больше номер строки, тем короче отступ: в первой строке он максимальный ($x - 1$), в последней ($i = x$) — нулевой. Ширина одного элемента отступа равна ширине выводимого элемента "*" (два пробела), поэтому треугольник выровнен по правому краю.
3. Второй внутренний цикл выполняется i раз и выводит i символов * (каждый с пробелом) — количество звёздочек в строке совпадает с номером строки.
4. После внутренних циклов выводится перевод строки.

Функция `main`: читает целое число с проверкой ввода и требует $x > 0$. Затем вызывается `RightTriangle(x)`.

Тестирование

```
• wakezup@pc:~/labs/lab1$ ./3_9
[ВВОД] Введите целое положительное число: 3
[РЕЗУЛЬТАТ] Прямоугольный треугольник с 3 строками:
  *
 * *
* * *

• wakezup@pc:~/labs/lab1$ ./3_9
[ВВОД] Введите целое положительное число: 4
[РЕЗУЛЬТАТ] Прямоугольный треугольник с 4 строками:
  *
 * *
* * *
* * * *

• wakezup@pc:~/labs/lab1$ ./3_9
[ВВОД] Введите целое положительное число: -1
[ОШИБКА] Введенное значение не является целым положительным числом!
❖ wakezup@pc:~/labs/lab1$
```

Задание 4. Массивы

В этом задании массивы создаются динамически (`new int[length]`) и освобождаются оператором `delete[]` на всех ветках выхода из программы. Длина массива передаётся в функции отдельным параметром, потому что при передаче в функцию массив превращается в указатель и теряет информацию о размере. По этой причине сигнатуры функций дополнены параметрами длины по сравнению с условием.

Задача 1. Поиск первого значения

Текст задачи

Дана сигнатура функции: `int findFirst (int arr[], int x);`

Необходимо реализовать функцию таким образом, чтобы она возвращала индекс первого вхождения числа `x` в массив `arr`. Если число не входит в массив – возвращается `-1`.

Пример:

`arr=[1,2,3,4,2,2,5]`

`x=2`

результат: 1

Алгоритм решения

Функция `int FindFirst(int* arr, const int length, int x)`:

1. Цикл `for` перебирает индексы `i` от 0 до `length - 1` в порядке возрастания.
2. Если `arr[i] == x`, функция немедленно возвращает `i`: так как обход идёт с начала, найденный индекс — индекс первого вхождения.
3. Если цикл завершился и совпадений не было, возвращается `-1`.

Длина массива передаётся отдельным параметром `length`: при передаче в функцию массив превращается в указатель и информация о его размере теряется.

Функция `main`:

1. Читает длину массива и проверяет, что это положительное целое число.
2. Выделяет память: `int* arr = new int[length]`.
3. Читает `length` элементов (каждый — с проверкой ввода), затем проверяет, что в строке нет лишних значений.
4. Читает искомое число `x`.
5. Вызывает `FindFirst` и выводит позицию либо сообщение «не найдено».
6. Освобождает память `delete[] arr` — в том числе перед каждым досрочным выходом по ошибке ввода.

Тестирование

```
• wakezup@pc:~/labs/lab1$ ./4_1
[ВВОД] Введите длину массива: 7
[ВВОД] Введите 7 чисел для заполнения массива (каждое число с новой строки):
1
2
3
4
2
2
5
[ВВОД] Введите число для поиска в массиве: 2
[РЕЗУЛЬТАТ] Число 2 найдено в массиве на позиции 1.
• wakezup@pc:~/labs/lab1$ ./4_1
[ВВОД] Введите длину массива: 4
[ВВОД] Введите 4 чисел для заполнения массива (каждое число с новой строки):
1
2
3
4
[ВВОД] Введите число для поиска в массиве: 5
[РЕЗУЛЬТАТ] Число 5 не найдено в массиве.
• wakezup@pc:~/labs/lab1$ ./4_1
[ВВОД] Введите длину массива: 0
[ОШИБКА] Введенное значение не является положительным целым числом!
❖ wakezup@pc:~/labs/lab1$
```

Задача 3. Поиск максимального

Текст задачи

Дана сигнатура функции: `int maxAbs (int arr[]);`

Необходимо реализовать функцию таким образом, чтобы она возвращала наибольшее по модулю (то есть без учета знака) значение массива `arr`.

Пример:

`arr=[1,-2,-7,4,2,2,5]`

результат: -7

Алгоритм решения

Функция `int MaxAbs(int* arr, const int length):`

1. В переменную `max_abs_value` записывается модуль первого элемента: значение `arr[0]`, а если оно отрицательное — то же значение с обратным знаком.
2. Цикл `for` проходит элементы со второго (`i = 1`) до последнего. Для каждого элемента вычисляется его модуль `current_abs` (отрицательное число меняет знак).
3. Если `current_abs > max_abs_value`, то `max_abs_value` обновляется.
4. После цикла возвращается `max_abs_value` — наибольшее значение модуля среди элементов массива.

Функция работает за один проход по массиву. Длина массива передаётся отдельным параметром `length`.

Функция `main`: читает длину массива (положительное целое), выделяет память `new int[length]`, читает элементы с проверкой каждого ввода и проверкой отсутствия лишних значений, вызывает `MaxAbs` и выводит результат. Память освобождается `delete[] arr` на всех ветках выхода.

Тестирование

```
• wakezup@pc:~/labs/lab1$ ./4_3
[ВВОД] Введите длину массива: 5
[ВВОД] Введите 5 чисел для заполнения массива (каждое число с новой строки):
1
-4
2
3
-2
[РЕЗУЛЬТАТ] Максимальное по модулю число в массиве равно 4.
• wakezup@pc:~/labs/lab1$ ./4_3
[ВВОД] Введите длину массива: -1
[ОШИБКА] Введенное значение не является положительным целым числом!
• wakezup@pc:~/labs/lab1$
```

Задача 5. Добавление массива в массив

Текст задачи

Дана сигнатура функции: `int * add (int arr[], int ins[], int pos);`

Необходимо реализовать функцию таким образом, чтобы она возвращала новый массив, который будет содержать все элементы массива `arr`, однако в позицию `pos` будут вставлены значения массива `ins`.

Пример:

arr=[1,2,3,4,5]

ins=[7,8,9]

pos=3

результат: [1,2,3,7,8,9,4,5]

Алгоритм решения

Функция `int* Add(int* arr, int* ins, const int length, const int ins_length, int pos)`

создаёт новый массив и заполняет его в три прохода:

1. Выделяется память под новый массив размером `length + ins_length`.
2. Первый цикл копирует в начало нового массива элементы `arr[0] ... arr[pos - 1]` — всё, что стоит до позиции вставки.
3. Второй цикл копирует все `ins_length` элементов массива `ins` в позиции `pos ... pos + ins_length - 1`.
4. Третий цикл копирует оставшиеся элементы `arr[pos] ... arr[length - 1]`, сдвигая их вправо на `ins_length` позиций: `new_arr[ins_length + i] = arr[i]`.
5. Функция возвращает указатель на новый массив. Исходные массивы не изменяются.

Размеры исходного и вставляемого массивов передаются отдельными параметрами, так как по указателю их определить нельзя.

Функция `main`: читает длину и элементы исходного массива, затем длину и элементы вставляемого массива (каждый ввод проверяется; длины должны быть положительными), затем позицию вставки. Позиция считается допустимой, если $0 \leq \text{pos} \leq \text{length}$. После вызова `Add` новый массив выводится на экран, а вся динамическая память (`new_arr`, `arr`, `ins`) освобождается через `delete[]`.

Тестирование

```
• wakezup@pc:~/labs/lab1$ ./4_5
[ВВОД] Введите длину исходного массива: 3
[ВВОД] Введите 3 чисел для заполнения исходного массива (каждое число с новой строки):
1
2
3
[ВВОД] Введите длину вставляемого массива: 2
[ВВОД] Введите 2 чисел для заполнения вставляемого массива (каждое число с новой строки):
7
8
[ВВОД] Введите позицию для вставки: 2
[РЕЗУЛЬТАТ] Новый массив: 1 2 7 8 3
• wakezup@pc:~/labs/lab1$ ./4_5
[ВВОД] Введите длину исходного массива: 3
[ВВОД] Введите 3 чисел для заполнения исходного массива (каждое число с новой строки):
1
2
3
[ВВОД] Введите длину вставляемого массива: 2
[ВВОД] Введите 2 чисел для заполнения вставляемого массива (каждое число с новой строки):
7
8
[ВВОД] Введите позицию для вставки: 0
[РЕЗУЛЬТАТ] Новый массив: 7 8 1 2 3
⊙ wakezup@pc:~/labs/lab1$ ./4_5
[ВВОД] Введите длину исходного массива: 3
[ВВОД] Введите 3 чисел для заполнения исходного массива (каждое число с новой строки):
1
2
3
[ВВОД] Введите длину вставляемого массива: 2
[ВВОД] Введите 2 чисел для заполнения вставляемого массива (каждое число с новой строки):
7
8
[ВВОД] Введите позицию для вставки: 5
[ОШИБКА] Введенное значение не является допустимой позицией!
❖ wakezup@pc:~/labs/lab1$
```

Задача 7. Возвратный реверс

Текст задачи

Дана сигнатура функции: `int * reverseBack (int arr[]);`

Необходимо реализовать функцию таким образом, чтобы она возвращала новый массив, в котором значения массива `arr` записаны задом наперед.

Пример:

`arr=[1,2,3,4,5]`

результат: `[5,4,3,2,1]`

Алгоритм решения

Функция `int* ReverseBack(int* arr, const int length):`

1. Выделяется память под новый массив `reversed_arr` той же длины `length`.
2. Цикл `for` по `i` от 0 до `length - 1` записывает в `i`-ю позицию нового массива элемент исходного массива с «зеркальным» индексом: `reversed_arr[i] = arr[length - 1 - i]`. Первый элемент нового массива берётся с конца исходного, второй — с предпоследней позиции и так далее.
3. Функция возвращает указатель на новый массив. Исходный массив остаётся без изменений.

Функция `main`: читает длину массива и его элементы с проверкой ввода, вызывает `ReverseBack`, выводит элементы нового массива через запятую и освобождает обе области памяти (`reversed_arr` и `arr`) оператором `delete[]`.

Тестирование

```
• wakezup@pc:~/labs/lab1$ ./4_7
[ВВОД] Введите длину массива: 3
[ВВОД] Введите 3 чисел для заполнения массива (каждое число с новой строки):
1
2
3
[РЕЗУЛЬТАТ] Перевернутый массив: 3, 2, 1
• wakezup@pc:~/labs/lab1$ ./4_7
[ВВОД] Введите длину массива: 1
[ВВОД] Введите 1 чисел для заполнения массива (каждое число с новой строки):
1
[РЕЗУЛЬТАТ] Перевернутый массив: 1
• wakezup@pc:~/labs/lab1$ ./4_7
[ВВОД] Введите длину массива: 0
[ОШИБКА] Введенное значение не является положительным целым числом!
❖ wakezup@pc:~/labs/lab1$
```

Задача 9. Все вхождения

Текст задачи

Дана сигнатура функции: `int * findAll (int arr[], int x);`

Необходимо реализовать функцию таким образом, чтобы она возвращала новый массив, в котором записаны индексы всех вхождений числа `x` в массив `arr`.

Пример:

arr=[1,2,3,8,2,2,9]

x=2

результат: [1,4,5]

Алгоритм решения

Функция `int* FindAll(int* arr, const int length, int target, int& count)`: размер результата заранее неизвестен, поэтому поиск выполняется в два прохода.

1. Первый проход считает количество вхождений `target` в массив и сохраняет его в параметре-ссылке `count`. Через этот параметр вызывающая функция узнаёт длину возвращённого массива.
2. Если вхождений нет (`count == 0`), функция возвращает `nullptr`.
3. Иначе выделяется массив `indices` размером `count`.
4. Второй проход снова просматривает исходный массив и при каждом совпадении записывает индекс `i` в очередную свободную ячейку `indices` (`indices[index++] = i`).
5. Функция возвращает указатель на массив индексов.

Функция `main`: читает длину массива и его элементы с проверкой ввода, затем искомое число. Если `FindAll` вернула `nullptr`, выводится сообщение «не найдено»; иначе выводятся `count` индексов через запятую, и память `indices` освобождается через `delete[]`. Исходный массив `arr` освобождается в любом случае.

Тестирование

```
● wakezup@pc:~/labs/lab1$ ./4_9
[ВВОД] Введите длину массива: 7
[ВВОД] Введите 7 чисел для заполнения массива (каждое число с новой строки):
1
2
3
8
2
2
9
[ВВОД] Введите число для поиска в массиве: 2
[РЕЗУЛЬТАТ] Число 2 найдено в массиве на позициях: 1, 4, 5.
● wakezup@pc:~/labs/lab1$ ./4_9
[ВВОД] Введите длину массива: 3
[ВВОД] Введите 3 чисел для заполнения массива (каждое число с новой строки):
1
2
3
[ВВОД] Введите число для поиска в массиве: 4
[РЕЗУЛЬТАТ] Число 4 не найдено в массиве.
⊙ wakezup@pc:~/labs/lab1$ ./4_9
[ВВОД] Введите длину массива: 3
[ВВОД] Введите 3 чисел для заполнения массива (каждое число с новой строки):
1
a
[ОШИБКА] Введенное значение не является целым числом!
❖ wakezup@pc:~/labs/lab1$
```